

ShopEase

Mini E-Commerce Web App with Interactive Shopping Cart

Beginner Capstone Project - Internship Submission

Built with HTML, CSS and Vanilla JavaScript

Submitted by: _____ Date: _____

1. Project Overview

ShopEase is a single-page product catalog with a working shopping cart. Products are stored as a JavaScript array of objects and rendered dynamically into the page. Users can search and filter items, add them to a cart, change quantities, see a live item count and total price, and review the order in a modal checkout preview before confirming.

The project needs no libraries, frameworks or build tools. It is one **index.html** file that runs by double-clicking it, which makes it easy to review and to host on GitHub Pages.

2. Key Features

- **Dynamic rendering** - product cards and cart rows are generated from JS objects using template literals.
- **Cart logic** - add items, increase or decrease quantity, remove items; the cart is an object of id to quantity.
- **Live totals** - item count badge in the header and the total price recalculate on every change.
- **Search and category filter** - re-render the catalog instantly as the user types or selects.
- **Modal checkout preview** - itemised order summary, Confirm and Back buttons, then a thank-you state.
- **Responsive design** - CSS Grid layout that collapses to one column on phones.
- **Edge cases handled** - empty cart message, disabled checkout button, no-results message.

3. Technology Stack

Layer	Technology	Purpose
Structure	HTML5	Semantic page skeleton and containers
Styling	CSS3 (Grid, Flexbox, variables)	Responsive layout, cards, modal, hover effects
Logic	Vanilla JavaScript (ES6)	State, rendering, events, calculations

4. How It Works

Data

The **products** array holds objects with id, name, category, price and colour. Adding a new product only requires adding one object - the UI updates automatically.

State

A single variable, **cart**, maps product ids to quantities. Count and total are derived from it with `reduce()`, so there is one source of truth and no duplicated data.

Rendering

`renderProducts()` filters the array and builds the cards; `renderCart()` rebuilds the cart panel, total, badge and checkout button state.

Events

Event delegation is used: one click listener on the product grid and one on the cart panel read data - * attributes, so newly rendered buttons work without re-binding.

Checkout modal

`openCheckout()` builds an order summary and shows the overlay. `confirmOrder()` clears the cart and shows a success message. Clicking the dark backdrop also closes the modal.

5. How to Run

- Save the code from Section 8 as **index.html** (or use the file supplied with this report).
- Double-click the file to open it in any modern browser.
- Optional: upload it to a GitHub repository and enable GitHub Pages for a live demo link.

6. Testing Checklist

- Add the same product twice - quantity becomes 2 and total doubles.
- Press minus at quantity 1 - the item is removed.
- Empty the cart - the checkout button is disabled and the total shows zero.
- Search for a word that matches nothing - the no-results message appears.
- Confirm an order - the cart resets and the badge shows 0 items.

7. Learning Outcomes and Future Improvements

Skills shown: DOM manipulation, array methods (map, filter, reduce, find), template literals, event delegation, state-driven rendering, responsive CSS.

Next steps: persist the cart with `localStorage`, add a discount-code field, product images, sort by price, and a form for shipping details with validation.

8. Complete Source Code (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ShopEase - Mini E-Commerce App</title>
<style>
:root { --brand:#4f46e5; --bg:#f5f6fa; --card:#fff; --text:#1f2937; --muted:#6b7280; }
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: system-ui, sans-serif; background: var(--bg); color: var(--text); }
header { background: var(--brand); color: #fff; padding: 14px 24px; display: flex;
  justify-content: space-between; align-items: center; position: sticky;
  top: 0; z-index: 5; }
header h1 { font-size: 1.3rem; }
.cart-btn { background: #fff; color: var(--brand); border: 0; padding: 8px 16px;
  border-radius: 20px; font-weight: 700; cursor: pointer; }
.layout { display: grid; grid-template-columns: 1fr 320px; gap: 20px; padding: 20px;
  max-width: 1150px; margin: auto; }
.toolbar { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 16px; }
.toolbar input, .toolbar select { padding: 9px 12px; border: 1px solid #d1d5db;
  border-radius: 8px; font-size: .95rem; }
.toolbar input { flex: 1; min-width: 160px; }
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(190px, 1fr)); gap: 16px; }
.card { background: var(--card); border-radius: 12px; overflow: hidden;
  box-shadow: 0 2px 8px rgba(0,0,0,.06); transition: transform .2s; }
.card:hover { transform: translateY(-4px); }
.thumb { height: 110px; display: flex; align-items: center; justify-content: center;
  font-size: 2.6rem; font-weight: 800; color: #fff; }
.info { padding: 12px; }
.info h3 { font-size: 1rem; }
.cat { color: var(--muted); font-size: .8rem; margin: 2px 0 8px; }
.row { display: flex; justify-content: space-between; align-items: center; }
.price { font-weight: 700; }
button.add { background: var(--brand); color: #fff; border: 0; padding: 7px 12px;
  border-radius: 8px; cursor: pointer; }
button.add:hover { opacity: .85; }
aside { background: var(--card); border-radius: 12px; padding: 16px; height: fit-content;
  position: sticky; top: 80px; box-shadow: 0 2px 8px rgba(0,0,0,.06); }
aside h2 { margin-bottom: 12px; }
.item { display: flex; justify-content: space-between; align-items: center;
  padding: 8px 0; border-bottom: 1px solid #eee; font-size: .9rem; }
.qty button { width: 24px; height: 24px; border: 1px solid #d1d5db; background: #fff;
  border-radius: 6px; cursor: pointer; }
.qty span { margin: 0 6px; }
.rm { background: none; border: 0; color: #dc2626; cursor: pointer; margin-left: 6px; }
.total { display: flex; justify-content: space-between; font-weight: 700;
  margin: 14px 0; font-size: 1.1rem; }
.checkout { width: 100%; padding: 11px; background: #16a34a; color: #fff; border: 0;
  border-radius: 8px; font-size: 1rem; cursor: pointer; }
.checkout:disabled { background: #9ca3af; cursor: not-allowed; }
.empty { color: var(--muted); text-align: center; padding: 20px 0; }
.overlay { position: fixed; inset: 0; background: rgba(0,0,0,.5); display: none;
  align-items: center; justify-content: center; z-index: 10; }
.overlay.open { display: flex; }
.modal { background: #fff; border-radius: 14px; padding: 24px; width: min(440px, 92%);
  max-height: 85vh; overflow: auto; }
.modal h2 { margin-bottom: 12px; }
.modal .actions { display: flex; gap: 10px; margin-top: 16px; }
.modal .actions button { flex: 1; padding: 10px; border-radius: 8px; border: 0;
  cursor: pointer; font-size: .95rem; }
.btn-cancel { background: #e5e7eb; }
.btn-confirm { background: var(--brand); color: #fff; }
@media (max-width: 800px) { .layout { grid-template-columns: 1fr; } aside { position: static; } }
</style>
</head>
<body>
<header>
  <h1>ShopEase</h1>
  <button class="cart-btn" id="cartBadge">Cart: 0 items</button>
</header>

<div class="layout">
  <main>
    <div class="toolbar">
```

```

        <input id="search" type="search" placeholder="Search products...">
        <select id="category"></select>
    </div>
    <div class="grid" id="products"></div>
</main>
<aside>
    <h2>Your Cart</h2>
    <div id="cartItems"></div>
    <div class="total"><span>Total</span><span id="cartTotal"></span></div>
    <button class="checkout" id="checkoutBtn">Checkout</button>
</aside>
</div>

<div class="overlay" id="overlay">
    <div class="modal" id="modal"></div>
</div>

<script>
// ----- DATA -----
const CURRENCY = '\u20B9';
const products = [
    { id: 1, name: 'Wireless Earbuds', category: 'Audio', price: 1999, color: '#6366f1' },
    { id: 2, name: 'Bluetooth Speaker', category: 'Audio', price: 2499, color: '#8b5cf6' },
    { id: 3, name: 'Smart Watch', category: 'Wearables', price: 3999, color: '#0ea5e9' },
    { id: 4, name: 'Fitness Band', category: 'Wearables', price: 1499, color: '#14b8a6' },
    { id: 5, name: 'Laptop Stand', category: 'Accessories', price: 899, color: '#f59e0b' },
    { id: 6, name: 'USB-C Hub', category: 'Accessories', price: 1299, color: '#ef4444' },
    { id: 7, name: 'Mechanical Keyboard', category: 'Accessories', price: 3499, color: '#ec4899' },
    { id: 8, name: 'Gaming Mouse', category: 'Accessories', price: 1799, color: '#10b981' }
];

// cart: { productId: quantity }
let cart = {};

// ----- HELPERS -----
const $ = (id) => document.getElementById(id);
const money = (n) => CURRENCY + n.toLocaleString('en-IN');
const findProduct = (id) => products.find((p) => p.id === id);
const cartCount = () => Object.values(cart).reduce((a, b) => a + b, 0);
const cartTotal = () =>
    Object.entries(cart).reduce((sum, [id, q]) => sum + findProduct(+id).price * q, 0);

// ----- RENDER PRODUCTS -----
function renderCategories() {
    const cats = ['All', ...new Set(products.map((p) => p.category))];
    $('category').innerHTML = cats.map((c) => `<option>${c}</option>`).join('');
}

function renderProducts() {
    const term = $('search').value.toLowerCase();
    const cat = $('category').value;
    const list = products.filter((p) =>
        (cat === 'All' || p.category === cat) && p.name.toLowerCase().includes(term));

    $('products').innerHTML = list.length
        ? list.map((p) => `
            <div class="card">
                <div class="thumb" style="background:${p.color}">${p.name[0]}</div>
                <div class="info">
                    <h3>${p.name}</h3>
                    <p class="cat">${p.category}</p>
                    <div class="row">
                        <span class="price">${money(p.price)}</span>
                        <button class="add" data-id="${p.id}">Add</button>
                    </div>
                </div>
            </div>
        `).join('')
        : `<p class="empty">No products found.</p>`;
}

// ----- RENDER CART -----
function renderCart() {
    const entries = Object.entries(cart);
    $('cartItems').innerHTML = entries.length
        ? entries.map(([id, q]) => {
            const p = findProduct(+id);
            return `

```

```

        <div class="item">
            <div><strong>${p.name}</strong><br>${money(p.price)}</div>
            <div class="qty">
                <button data-act="dec" data-id="${id}">-</button>
                <span>${q}</span>
                <button data-act="inc" data-id="${id}">+</button>
                <button class="rm" data-act="rm" data-id="${id}">x</button>
            </div>
        </div>`;
    }).join('')
    : '<p class="empty">Your cart is empty.</p>';

    $('cartTotal').textContent = money(cartTotal());
    const n = cartCount();
    $('cartBadge').textContent = `Cart: ${n} item${n === 1 ? '' : 's'}`;
    $('checkoutBtn').disabled = n === 0;
}

// ----- CART ACTIONS -----
function addToCart(id) { cart[id] = (cart[id] || 0) + 1; renderCart(); }
function changeQty(id, delta) {
    cart[id] = (cart[id] || 0) + delta;
    if (cart[id] <= 0) delete cart[id];
    renderCart();
}
function removeItem(id) { delete cart[id]; renderCart(); }

// ----- MODAL -----
function openCheckout() {
    const rows = Object.entries(cart).map(([id, q]) => {
        const p = findProduct(+id);
        return `<div class="item"><span>${p.name} x ${q}</span>
            <span>${money(p.price * q)}</span></div>`;
    }).join('');
    $('modal').innerHTML = `
        <h2>Order Preview</h2>${rows}
        <div class="total"><span>Total (${cartCount()} items)</span>
        <span>${money(cartTotal())}</span></div>
        <div class="actions">
            <button class="btn-cancel" id="cancelBtn">Back</button>
            <button class="btn-confirm" id="confirmBtn">Confirm Order</button>
        </div>`;
    $('overlay').classList.add('open');
    $('cancelBtn').onclick = closeModal;
    $('confirmBtn').onclick = confirmOrder;
}

function confirmOrder() {
    cart = {};
    renderCart();
    $('modal').innerHTML = `
        <h2>Thank you!</h2>
        <p>Your order has been placed successfully.</p>
        <div class="actions">
            <button class="btn-confirm" id="doneBtn">Continue Shopping</button>
        </div>`;
    $('doneBtn').onclick = closeModal;
}

function closeModal() { $('overlay').classList.remove('open'); }

// ----- EVENTS -----
$('products').addEventListener('click', (e) => {
    if (e.target.classList.contains('add')) addToCart(+e.target.dataset.id);
});
$('cartItems').addEventListener('click', (e) => {
    const { act, id } = e.target.dataset;
    if (act === 'inc') changeQty(+id, 1);
    if (act === 'dec') changeQty(+id, -1);
    if (act === 'rm') removeItem(+id);
});
$('search').addEventListener('input', renderProducts);
$('category').addEventListener('change', renderProducts);
$('checkoutBtn').addEventListener('click', openCheckout);
$('overlay').addEventListener('click', (e) => { if (e.target === $('overlay')) closeModal(); });

// ----- INIT -----

```

```
    renderCategories();  
    renderProducts();  
    renderCart();  
</script>  
</body>  
</html>
```